ALLIANCE UNIVERSITY

Alliance School of Advanced Computing

STUDENT NAME

1. _____ (Reg. No.: _____)
2. _____ (Reg. No.: _____)
3. _____ (Reg. No.: _____)
4. _____ (Reg. No.: _____)
5. _____ (Reg. No.: _____)

Case study: Network Traffic Analysis Using Graph Theory

YEAR: 2026-2028

Department of Computer Applications

Program – Master of Computer Applications

Semester I

Submitted in partial fulfilment of the requirements for the Case Study component of the Master of Computer Applications (MCA) programme, Semester I.

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to everyone who supported and guided me throughout the successful completion of this case study on Network Traffic Analysis Using Graph Theory. The work required careful modelling of communication relationships, interpretation of discrete-mathematics concepts, and translation of those ideas into a practical security-operations narrative suitable for an MCA Semester I case study.

I am thankful to my Project Guide for providing valuable guidance, suggestions, encouragement and continuous support throughout the development of this case study. Discussions on modelling hosts as vertices, flows as edges, and attacks as topological patterns helped clarify both the academic and the operational value of the approach.

I also express my gratitude to the faculty members of the Alliance School of Advanced Computing and to my institution for providing laboratory resources, academic materials and a supportive learning environment. Access to networking and discrete-structures coursework made it possible to connect theory with a cybersecurity application.

I would also like to thank my friends and classmates for their cooperation, peer review and constructive feedback on tables, wording of the problem statement and practicality of the proposed detection pipeline. Their questions improved clarity and completeness.

Finally, I am deeply grateful to my family for their constant encouragement, motivation and support during the research and writing process. Their patience made sustained focus possible across data-collection design, analysis and solution chapters.

I sincerely thank everyone who contributed directly or indirectly to the successful completion of this case study. Any remaining errors or omissions are my own responsibility.

This acknowledgement also recognises the broader academic framing of the work: applying graph theory from discrete structures to a cybersecurity case that emphasises evidence tables, reasoned analysis and a phased operational solution suitable for evaluation under the Alliance University MCA Semester I case-study guidelines.

Case Study Name: Network Traffic Analysis Using Graph Theory

Team Name: _____

Signature of Students

1. _____ 2. _____ 3. _____
4. _____ 5. _____

Signature of Faculty & Project Guide

_____

Project Faculty & Project Guide Name: Abhishek A

2. Problem Statement

Why network traffic needs graph-based structural analysis

Background. Modern enterprise, campus and cloud networks carry huge volumes of communication among servers, workstations, mobiles, cloud services and Internet of Things devices. Each conversation leaves a relationship: who talked to whom, when, on which protocol, and with what volume. Security teams already collect NetFlow, firewall logs and IDS alerts, yet those records are often examined host by host or as isolated volume spikes. The relational structure of communication is underused even though it is exactly the structure that many attacks rearrange.

In a typical mid-to-large environment, more than a thousand IP addresses may be active in a busy hour. Help desks and antivirus tools still report malware and phishing, but many modern campaigns—distributed denial of service, wide scanning, botnet command-and-control and lateral movement—are easier to see as stars, hubs and unusual paths than as single signature matches. Graph theory supplies a precise language for those patterns.

The problem. Organisations still rely mainly on signature IDS, volume thresholds and static firewall policy. Each control helps, yet each fails when attackers distribute activity, encrypt payloads, or keep byte rates intentionally low. Without a topology baseline of normal who-talks-to-whom behaviour, structural drift is noticed too late and SOC analysts receive alerts that lack ranked hosts or a local subgraph for triage.

Main analysis problems

Problem	What is happening	Why it matters
Volume-only monitoring	Alerts on bytes/packets; busy hours look like attacks	High false positives; slow attacks missed
Signature dependence	IDS matches only known patterns and IoCs	Zero-day and encrypted C2 evade detection
Host-centric view	Each IP examined alone, not as part of a graph	Coordinated multi-host attacks stay hidden
No topology baseline	Normal who-talks-to-whom pattern not modelled	Structural drift is noticed too late
Weak localization	Alerts lack ranked hosts and local subgraphs	Analysts waste hours on triage
Encrypted payloads	HTTPS hides content; flow relationships unused	Lost chance to use available metadata

Findings from the problem table. Volume-only rules confuse legitimate busy periods with attacks and miss low-and-slow campaigns. Signature tools cannot see novel or encrypted command channels. Host-centric investigation hides coordinated behaviour that only appears when edges among many hosts are viewed together. Without a topology baseline, gradual compromise looks like normal growth until damage is already large.

Implications for this case study. A graph-theoretic approach treats IP hosts as vertices and observed flows as edges in fixed time windows. Degree, centrality, communities and entropy then become security features. The same metadata used for capacity planning becomes evidence for detecting scanners, DDoS stars, botnet hubs and unusual internal pivots—even when payloads remain encrypted.

Problem statement

"Organisations lack a structured, graph-theoretic approach to analysing network traffic. As a result, distributed and stealthy threats such as DDoS, scanning, botnets and lateral movement remain harder to detect, localise and explain than volume or signature tools alone can achieve."

Scope of the study

- Models traffic as graphs: nodes = IP hosts; edges = flows or connections in a time window.
- Uses flow metadata (NetFlow/IPFIX/Zeek/PCAP summaries), not decrypted payloads.
- Applies degree, centrality, communities, graphlets and entropy for anomaly detection.
- Focuses on cyber communication structure; physical building security is out of scope.
- Proposes a collect–analyse–detect–respond pipeline with tools, phases and limits.

Impact if the problem is not solved

- Security: scanners, botnets and lateral movement remain invisible longer and spread deeper.
- Operational: DDoS and ransomware recovery take longer without a clear attack topology.
- Analytical: SOC teams drown in volume alerts without structural prioritisation.
- Business: longer downtime, higher breach risk and loss of customer trust.

Observation. Solving the problem does not replace firewalls, EDR or IDS; it adds a structural layer that explains who is connected to whom when an alert fires. That layer is the focus of the remaining sections.

3. Objectives

What this study aims to achieve

Main objective

To study how graph theory can be applied to network traffic analysis, identify structural signals of cyberattacks, and recommend a practical detection pipeline that improves anomaly visibility, localisation and explainability for Security Operations Centre (SOC) teams.

Specific objectives

No.	Objective	How it is achieved	Section
1	Collect suitable traffic data	NetFlow/PCAP/logs; windowed graphs	Data Collection
2	Build traffic graphs correctly	IPs as nodes; flows as weighted edges	Data Collection
3	Apply graph measures	Degree, centrality, communities, graphlets, entropy	Analysis
4	Link attacks to topology	DDoS, scan, botnet, lateral movement patterns	Analysis
5	Design detection pipeline	Baseline, scoring, IP localization, response	Solution
6	Assess benefits and limits	Outcomes, tools, constraints, phased plan	Solution, Conclusion

These objectives follow institutional case-study logic: clear problem → measurable aims → evidence → analysis → prioritised solution → management recommendations. Each objective maps to later tables so the reader can verify that data collection, analysis and solution chapters close the loop opened here.

Findings. Objectives 1–2 ensure the graph is built from realistic telemetry rather than abstract drawings. Objectives 3–4 convert discrete-structures concepts into attack-relevant features. Objectives 5–6 force the study to remain operational: a pipeline, tool stack, benefit estimate and honest limits—not only definitions.

Measurable targets (illustrative lab / deployment goals)

Indicator	Current (typical)	Target
Communication graph model in use	Not used / ad hoc only	Windowed graphs on monitored segments
High out-degree scanner detection	Often missed if low bytes	Flag within 1–2 time windows
DDoS structural (star) detection	Volume alert without topology	In-degree spike + star alert
False positives vs volume-only rules	High in busy periods	Lower when topology is normal
Time to name top suspect IPs	Manual multi-tool log search	Ranked IP list + local subgraph
Use of encrypted-traffic metadata	Payload-focused only	Full use of flow relationships
Attack-shape evidence in tickets	Rare / informal	TDG screenshot attached as standard

Implications. The targets are illustrative for a campus or SME SOC adopting graph analytics. They emphasise time-to-detect structural change, triage quality and metadata usefulness under encryption—not unrealistic zero-false-positive claims.

Importance of the study

- Gives management evidence to fund flow collection and graph analytics.
- Shows how discrete mathematics applies directly to cybersecurity practice.
- Helps SOC teams see attack shape—stars, hubs, communities—not only alert counts.
- Supports NIST CSF-style detect-and-respond thinking with a concrete data model.

Methodology

Step	Activity	Tool / source
1	Gather flow and log data	NetFlow/IPFIX, Zeek, PCAP, firewall/IDS
2	Slice windows and build graphs	Python/NetworkX, flow collector, CSV
3	Compute structural features	Degree, centrality, entropy, graphlets
4	Baseline compare; score anomalies	Thresholds / outlier models
5	Localize IPs and respond	Ranked lists, Gephi, SOC playbooks

The method starts tool-light so a student lab or small SOC can reproduce it, then adds heavier metrics (betweenness, graphlets, spectral features) after degree-based detection is stable. Sections 4 and 5 expand each step with tables and worked observations.

4.1 Data Collection

Sources, monitored environment profile and fields for graph construction

Graph-based analysis needs relationship telemetry more than full payloads. That is useful when TLS/HTTPS limits deep packet inspection. The minimum requirement is knowing which source spoke to which destination in a time window, optionally with ports, protocol, bytes and packets. Asset tags (workstation, server, IoT) enrich nodes without requiring decrypted content.

This case study uses the same source classes found in enterprise SOCs and academic cyber ranges. Table 1 maps each source to its role in a Traffic Dispersion Graph (TDG). Table 2 profiles an illustrative monitored environment. Table 3 lists essential fields extracted per flow before graph construction.

Table 1: Data sources for graph construction

Source	Key fields	Role in graph model
NetFlow / IPFIX	src/dst IP, ports, protocol, bytes, packets, time	Primary host-to-host edges
sFlow	sampled headers on high-speed links	Lightweight communication evidence
PCAP (Wireshark/tcpdump)	full headers; optional payload	Fine-grained validation graphs
Zeek / Suricata logs	connections, DNS, HTTP metadata	Typed edges and richer context
Firewall / IDS alerts	allow/deny, signatures, severity	Labels for evaluation and triage
DNS / proxy logs	client, domain, resolver	Bipartite host–domain graphs

Finding (Table 1). NetFlow/IPFIX and Zeek connection logs are sufficient to build useful TDGs at campus or SME scale. PCAP remains valuable for validation and forensics but is not required for every detection window. DNS and proxy logs add bipartite views that help explain unusual external destinations without decrypting HTTPS.

Observation. Mixing flow exporters with IDS labels creates a supervised evaluation path: graph features can be scored against known attack intervals even when the detector itself uses unsupervised baselines in production.

Table 2: Illustrative monitored environment profile

Segment	Approx. active IPs	Main traffic	Graph interest
User LAN	1,200	Web, DNS, email, LMS	Client degree patterns
Server VLAN	80	App, DB, file, auth	Hub / betweenness nodes
DMZ / VPN	40	Remote access, portals	External edge anomalies
IoT / printers / CCTV	150	Limited protocols	Unexpected scan behaviour
Total (peak window)	~1,470	Mixed enterprise mix	Full TDG snapshot

Finding (Table 2). User LAN dominates vertex count; servers dominate betweenness; VPN/DMZ edges are the natural place to watch for unusual external peers; IoT devices should stay low-degree and protocol-narrow. A sudden degree explosion on a camera or printer is a high-priority structural signal.

Table 3: Essential fields extracted per flow

Field	Example	Graph use
Time window	10:00:00–10:01:00	Selects snapshot G_t
Source IP	192.168.1.10	Edge start vertex
Destination IP	10.0.0.5	Edge end vertex
Protocol / port	TCP / 443	Optional edge label / filter
Bytes / packets	15240 / 28	Edge weight
Asset tag	Workstation / Server / IoT	Node colour and policy context

Observation: VPN and customer-facing segments usually create the most external edges. IoT and printer segments are normally low-degree and protocol-narrow. A sudden degree explosion on those devices is therefore a strong anomaly. Implications: collection design must retain src/dst identity, time and optional weights; asset tags turn raw graphs into actionable SOC pictures without needing payload inspection.

Data-quality note. Duplicate exporters, NAT boundaries and DHCP churn can inflate or merge vertices. Section 4.2 lists preprocessing rules that keep G_t comparable across windows so baselines remain meaningful.

Further implication. Choosing fields in Table 3 is a design decision with security consequences: omitting asset tags forces analysts to interpret raw IPs; omitting weights weakens exfiltration detection; omitting timestamps prevents windowed comparison. The collection plan should therefore be signed off by both network engineering and the SOC before detectors are tuned.

4.2 Data Collection

Windowing strategy, preprocessing quality and example records

A continuous flow stream must become a sequence of comparable graphs. Fixed time-window slicing does this. For each window t, flows inside the window form G_t = (V_t, E_t). Vertices are active IPs; edges are aggregated contacts (binary presence or weighted by bytes/packets). Comparing G_t with a baseline of recent clean windows reveals structural change.

Window length is a design choice. Short windows detect faster but are noisier. Long windows are stabler but slower. Table 4 summarises the trade-off recommended in this study.

Table 4: Windowing trade-offs

Window length	Advantage	Disadvantage	Recommended use
30 seconds	Very fast detection	Noisy, volatile graphs	Critical segments / attack labs
1 minute	Good speed–stability balance	Moderate noise	Campus / SME default
5 minutes	Stable topology for baselines	Slower reaction	Overnight trend learning

Finding (Table 4). A one-minute default balances speed and stability for campus networks. Thirty-second windows suit attack-range labs or critical VPN gateways. Five-minute windows help learn overnight baselines. Mixing lengths is acceptable: detect on short windows, explain trends on longer aggregates.

Preprocessing and data-quality steps

- Discard malformed, incomplete or duplicated flow records before building the graph.
- Normalise IPv4/IPv6 text forms; keep private vs public addressing consistent.
- Optionally filter broadcast/multicast when those edges are irrelevant to detection goals.
- Aggregate many flows between the same host pair into one weighted edge per window.
- Enrich nodes with subnet, external ASN/geo and asset role tags.
- When measuring accuracy, label known attack intervals and known attacker/victim IPs.

Table 5: Example processed edge record

Attribute	Value
Time window	10:00:00 – 10:00:30
Source IP	192.168.1.10
Destination IP	10.0.0.5
Protocol / destination port	TCP / 443
Bytes / packets	15240 / 28
Chosen edge weight	Bytes (alternative: binary contact = 1)
Node tags	src=Workstation; dst=Internal Web Server

Implication of Table 5. Binary contact edges emphasise peer diversity (excellent for scan detection). Byte-weighted edges emphasise intensity (helpful for exfiltration and DDoS). Many deployments compute both and use binary features for structure and weights for severity.

Table 6: Privacy and operational handling rules

Rule	Practice in this study design
Least privilege	Only SOC/analyst roles may query raw flows
Retention	Keep detailed flows only for the approved policy window
Academic datasets	Prefer anonymised campus traces or synthetic lab PCAP
Payload storage	Store full PCAP only when forensics require it
Sharing / publication	Anonymise internal IP maps before any external sharing

Observation: Graph analysis does not need decrypted HTTPS. Flow metadata alone builds TDGs and supports the structural features in Section 5. That keeps the method compatible with privacy expectations and modern encrypted traffic. Preprocessing quality is therefore as important as algorithm choice: dirty input graphs produce noisy centrality and false alerts.

Section link. With sources, windows and cleaned edge records defined, Section 5.1 selects graph measures; Section 5.2 maps attacks to shapes; Section 5.3 scores windows over time.

5.1 Analysis

Graph model and metrics that reveal traffic anomalies

With windowed graphs available, analysis measures structure. A graph G = (V, E) has vertices V and edges E. Here V is usually active IPs in the window and E is observed communication. Directed graphs keep request direction (useful for scan vs DDoS). Undirected graphs still reveal communities and hubs when direction is noisy or unavailable.

Graph theory fits because many attacks rearrange topology, not only volume. Questions become neighbourhood and path questions: Who became newly reachable? Which host bridges subnets it never bridged before? Did a star form into one victim or out from one scanner? Table 7 maps classical measures to security meaning.

Table 7: Graph measures and their security meaning

Measure	Traffic meaning	Typical anomaly signal
Degree / In / Out-degree	Number of distinct peers contacted	Scan: high out-degree; DDoS victim: high in-degree
Betweenness centrality	How often a host lies on shortest paths	Unexpected pivot / relay after compromise
Eigenvector / PageRank	Importance via important neighbours	Rising influence of a C2 hub
Clustering coefficient	How interconnected neighbours are	Change may indicate botnet formation
Communities / components	Groups of mutually related hosts	New isolated attack community appears
Graphlets / motifs	Counts of small subgraph patterns	Motif histogram shifts under attack
Degree-distribution entropy	Diversity of connectivity pattern	Sharp jump or collapse during DDoS
Spectral / Laplacian features	Global shape of graph spectrum	Heartbeat anomalies across windows

Finding (Table 7). Degree features are cheap, explainable and sufficient to catch many scanners and DDoS stars. Betweenness and communities add lateral-movement and botnet visibility. Graphlets and spectral features refine detection after cheap filters fire. A two-stage design keeps computation practical on larger graphs.

Operational implication. In operations, cheap features (degree, new-edge count, simple entropy) run on every window. Costly features (exact betweenness, large graphlet sets) run only on already-suspicious windows. Analysts receive both a score and a reason tied to a measure they can visualise.

Key points from the metric analysis

- Peer identity matters more than raw bytes for scanners and low-and-slow campaigns.
- Star topologies strongly indicate DDoS or aggressive scanning.
- Hub-and-spoke communities often indicate botnet command-and-control.
- New internal edges with rising betweenness suggest lateral movement.
- Explainable metrics fit SOC workflows better than opaque scores alone.

Conclusion of this page. Selecting measures is not an academic exercise only: each measure answers a triage question. Section 5.2 binds those measures to concrete attack classes so detection rules become explanations of adversary tactics.

Additional observation on baselines. Degree and entropy need a “normal” band learned from clean weeks of the same segment. Without that band, a popular web server’s high in-degree looks identical to a DDoS victim. Segment-aware baselines (User LAN vs Server VLAN vs IoT) therefore matter as much as the choice of metric. Asset tags collected in Section 4.1 make those segment baselines practical.

Additional implication for discrete structures coursework. The same definitions studied in graph theory—degree, path, component, adjacency—become operational features when V and E are taken from live NetFlow. That bridge between theory and SOC practice is a deliberate learning outcome of this MCA case study and should be visible in every later table that names a metric beside an attack class.

Worked note on directed versus undirected models. For scanning, out-degree on a directed graph is the natural signal. For community structure of botnet traffic, an undirected projection often suffices and is cheaper. Production pipelines may keep both views: directed for degree detectors and undirected for community modules. Analysts should know which view produced each alert so visualisation matches the score.

Summary takeaway for Section 5.1. Prefer explainable, cheap structural features first; escalate to richer centrality and motif analysis only when the cheap layer already flags a window. That discipline keeps both student labs and production SOCs within compute budgets while preserving the security meaning of every alert.

5.2 Analysis

Attack patterns mapped to graph structure (reason)

This section links adversary behaviour to graph shape. When the mapping is clear, detection rules stop being arbitrary thresholds and become explanations of tactics. Table 8 is the core reason table of the case study: each attack class has a distinctive topology and a primary feature set, plus a note on why volume-only monitoring fails.

Table 8: Attacks → topology → detection features

Attack	Graph shape	Primary features	Why volume alone fails
DDoS	Star into one victim	High in-degree, entropy shift	May look like a busy popular service
Port / network scan	Star from one scanner	High out-degree, many new edges	Each probe transfers few bytes
Botnet C2	Hub-and-spoke / small community	Community + hub centrality	HTTPS C2 looks like normal web
Lateral movement	New unusual internal paths	New edges, betweenness rise	Often low byte volume
Data exfiltration	Persistent heavy external edge	Weighted degree, destination novelty	May be intentionally slow

DDoS reason. Many attackers hit one victim; in-degree spikes and a star appears—even if each attacker is moderate in volume. Scanning reason. One source contacts many destinations with tiny probes; out-degree and new-edge counts explode while bytes stay small. Botnet reason. Many bots share one C2 hub; community detection and hub centrality rise while payloads look like ordinary HTTPS. Lateral-movement reason. Compromised hosts open new internal paths; betweenness of pivots rises before large data movement occurs.

Finding. The hardest cases for classical tools—distributed, encrypted or intentionally slow—are exactly those with the clearest topological footprints. That is the central analytical claim of this case study.

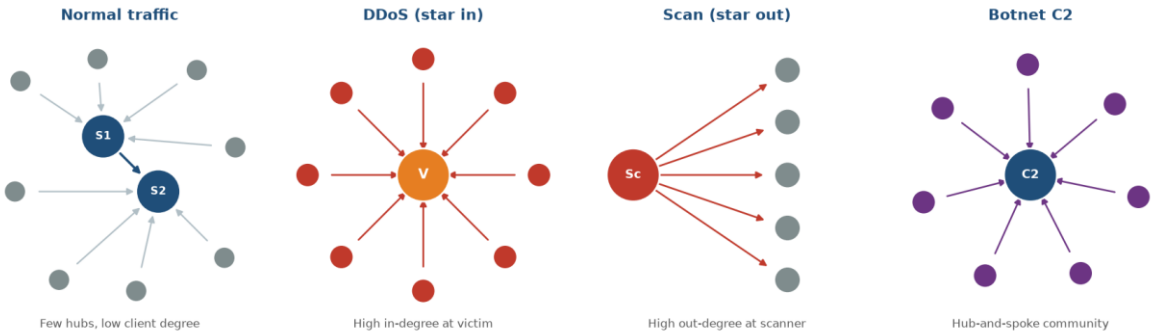


Figure 1. Traffic-graph shapes: normal communication, DDoS star, scanner star and botnet hub-and-spoke.

Table 9: Illustrative mini-scenario (campus IoT scan)

Time	Event	Graph observation	SOC action
14:10	Normal midday traffic	IoT camera out-degree ≈ 4	No alert
14:12	Camera compromised; scan starts	New edges into 10.0.0.0/16	Watch list
14:13–14:14	Scan widens rapidly	Out-degree rises toward ~900	High alert
14:14+	Detector localizes source	Top ranked IP = camera	Isolate + investigate

Why this example matters (reason)

- Individual probe flows were small, so a pure byte threshold likely stayed quiet.
- Degree centrality and new-edge counts changed sharply within two one-minute windows.
- The analyst received an alert, a ranked host and a visible star subgraph together.
- Structural change is therefore a stronger, signature-independent indicator for this threat class.

Conclusion of this page: attacks that are hardest for classical tools—distributed, encrypted or intentionally slow—are exactly those with the clearest topological footprints. That justifies investing in windowed TDGs before adding more opaque machine-learning layers.

Cross-check with the mini-scenario. In Table 9 the camera's byte contribution remained modest while its neighbour set exploded. A SOC relying only on bandwidth graphs would have seen a quiet device; a SOC with out-degree and new-edge monitors would have seen a star forming in two windows. That contrast is the practical reason this case study prefers structural features as first-line detectors for reconnaissance.

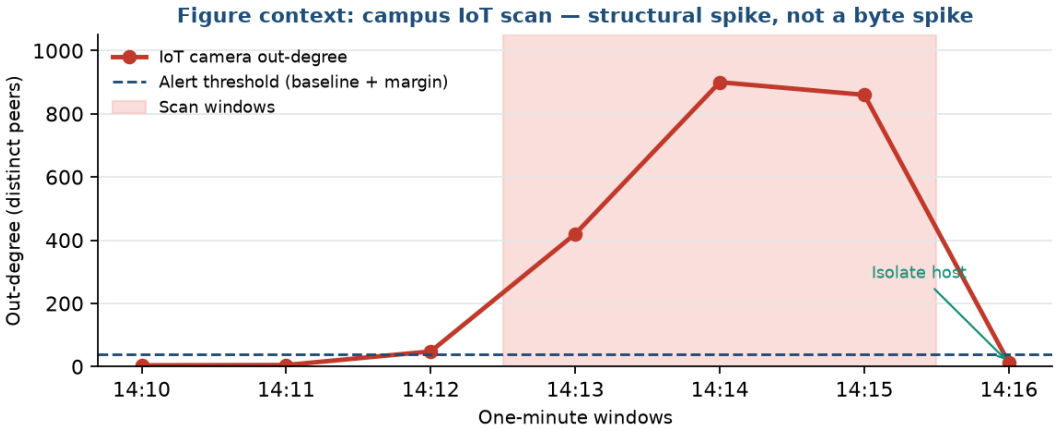


Figure 2. Out-degree of the compromised IoT camera across one-minute windows (Table 9 scenario).

Implication for playbooks. Each row of Table 8 should map to a short response card: what subgraph to open, which IP rank to trust first, and whether to isolate a host, rate-limit a segment or escalate to forensics. Without that mapping, graph alerts become another noisy queue. Section 6 therefore lists playbook-linked controls beside each High/Critical risk.

Observation on false positives. Not every star is malicious: a software-update mirror or a videoconference bridge can create high degree. Asset role tags and destination reputation reduce that confusion. The method still beats pure volume rules because it forces the analyst to ask “is this topology expected for this asset?” rather than only “are bytes high?”.

5.3 Analysis

Temporal scoring, baseline comparison and risk prioritisation

Attacks unfold over time, so analysis cannot stop at one static drawing. Each window yields a feature vector (max in-/out-degree, new edges, entropy, optional graphlets). The sequence of vectors is compared with a baseline learned from recent clean periods. Large deviations raise anomaly scores; stable busy traffic that stays inside the topological band stays quiet.

For anomalous windows, localization ranks IPs by contribution: largest degree delta, most new neighbours, or largest centrality jump. Analysts then open the local subgraph around the top-ranked hosts—often a star or unexpected bridge—and decide isolation, block or further forensics.

Table 10: Example anomaly scoring logic

Condition	Score effect	Likely label
Out-degree > μ + 5σ and many new destinations	High	Scanner / recon
In-degree spike; sources across many subnets/ASNs	High	DDoS victim
New dense community with one external hub	High	Botnet C2 pattern
Sudden internal cross-subnet edges on one host	Medium–High	Lateral movement
Rare persistent high-weight external destination	Medium–High	Possible exfiltration
Busy traffic but topology inside baseline band	Low	Benign load / expected peak

Finding (Table 10). Scoring is intentionally interpretable. Thresholds such as μ + 5σ are starting points for a lab; production systems may use robust z-scores, isolation forests or seasonal baselines. The important design choice is pairing score with a structural label and a ranked IP list.

Table 11: Priority risks addressed by graph analysis

ID	Risk	Likelihood	Impact	Score	Level
R1	Undetected scanning / recon	5	4	20	Critical
R2	DDoS without structural context	3	5	15	High
R3	Botnet C2 over encrypted channels	3	4	12	High
R4	Lateral movement inside LAN	3	5	15	High
R5	Alert fatigue from volume-only rules	4	3	12	High
R6	Slow exfiltration to rare destinations	2	4	8	Medium

Scoring uses Likelihood (1–5) × Impact (1–5). Levels: 1–5 Low, 6–11 Medium, 12–19 High, 20–25 Critical. Result: R1 is Critical; R2–R5 are High. These five risks lead the solution plan in Section 6. R6 remains Medium because volume is low but destination novelty and weighted edges still provide a usable signal.

Temporal views also support situational awareness. A daily “heartbeat” of max degree or entropy shows normal rhythms (morning logins, backup window, evening quiet) and highlights days when the whole graph shape drifts. Implications: graph analytics improve not only point alerts but also week-over-week security posture reporting to management.

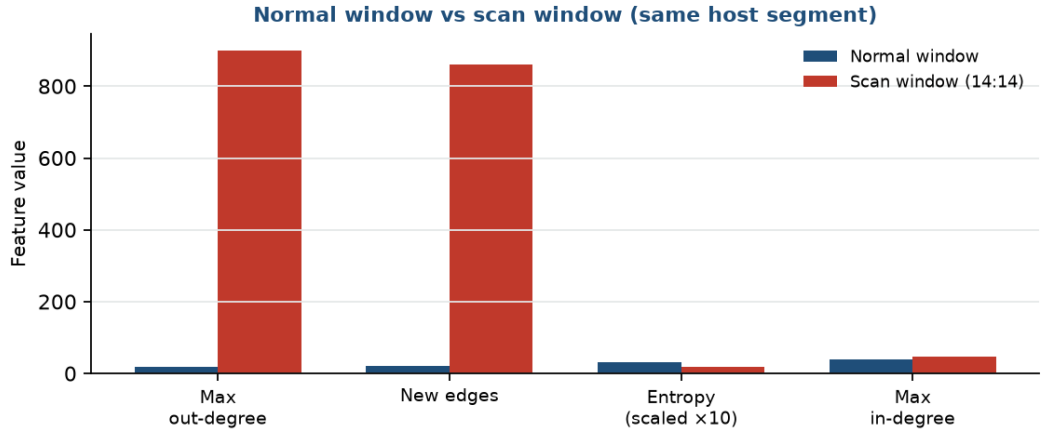


Figure 3. Structural features in a normal window versus the scan window — out-degree and new edges dominate.

Finding on risk ranking. Critical risk R1 (undetected scanning) scores highest because reconnaissance is frequent, cheap for attackers and often invisible to byte thresholds. High risks R2–R5 cover DDoS context, encrypted C2, lateral movement and alert fatigue—the exact failure modes of host-centric and volume-only programmes described in Section 2. Medium risk R6 remains visible through destination novelty and weighted edges.

Implication for Section 6. Solution spend and timeline should follow the score order: degree/new-edge detectors and baselines first (R1, R5), then star visualisation for DDoS (R2), then community and betweenness modules (R3, R4).

That ordering maximises early reduction of Critical exposure while keeping the student or SME implementation feasible.

Observation on localisation quality. Ranking IPs by contribution converts a vague “graph anomaly” into a shortlist an on-call engineer can act on. Combined with a one-hop or two-hop subgraph export, localisation is the difference between a research prototype and a SOC-ready control. Table 10’s labels exist to name the shortlist, not only to colour a dashboard.

Section bridge. With scores, labels and risk IDs defined, Section 6 converts R1–R5 into pipeline controls, tools, timelines and expected operational benefits without inventing unrelated recommendations.

6. Solution

Graph-based detection pipeline, controls, tools and expected benefit

The solution is a modular graph-based anomaly detection framework. Telemetry becomes windowed graphs; structural features are extracted; features are compared with a baseline; ranked hosts and local subgraphs support SOC response. Controls below map directly to Critical and High risks R1–R5 identified in Section 5.3.

Pipeline: Sensors/Routers/Zeek → NetFlow/Log collection → time-window slicing → build G_t → compute features (degree, centrality, graphlets, entropy) → compare with baseline → anomaly score + ranked IPs → visualisation (Gephi) → SOC ticket / block / isolate.

Table 12: Recommended pipeline controls

Problem addressed	Solution	Priority	Timeline
No traffic graph model	Enable NetFlow/Zeek + windowed TDG builder	High	Month 1
Missed scanners (R1)	Out-degree / new-edge detectors	High	Month 1–2
DDoS context (R2)	In-degree star detection + visualisation	High	Month 1–2
Encrypted C2 (R3)	Community + hub centrality alerts	High	Month 2–3
Lateral movement (R4)	Internal new-edge / betweenness watch	High	Month 2–3
Alert fatigue (R5)	Require structural change with volume	Medium	Month 2
Weak triage	Ranked IP list + Gephi local subgraph	Medium	Month 2–3
No baseline	Learn normal graph features from clean weeks	High	Month 1
Slow response	SOC playbooks linked to graph alert types	Medium	Month 3
Skill gap	Train analysts on TDGs and centrality ranks	Medium	Month 1–3

Finding (Table 12). Quick wins are flow export, baselines and degree detectors. Communities and betweenness follow once analysts trust the first alerts. Training is not optional: a star graph only helps if the on-call engineer can read it.

Table 13: Tool stack

Layer	Suggested tools
Collection	Zeek, nProbe, router NetFlow, Wireshark/tcpdump
Storage	CSV/SQLite (lab); flow collector / Elasticsearch (ops)
Graph compute	Python NetworkX; Graph-tool or Neo4j for larger graphs
Detection	Statistical thresholds; scikit-learn outlier models
Visualisation	Gephi or Cytoscape for stars, hubs and communities
Response	SIEM tickets, firewall blocks, host isolation playbooks

Table 14: Expected benefit (illustrative)

Measure	Expected value
Earlier scan detection	Within 1–2 windows via degree / new-edge spike
Clearer DDoS picture	Victim + top sources visible from star graph
False-positive reduction vs volume-only	Busy but topologically normal traffic stays quiet
Analyst triage time	Minutes with ranked IPs instead of hours of raw logs
Encrypted traffic usefulness	Metadata relationships remain fully usable

Benefit interpretation. Numbers are illustrative for a teaching and design case study, not a commercial ROI claim. Actual reductions depend on coverage of exporters, quality of baselines and analyst adoption. Even partial deployment of degree-based detectors already addresses Critical risk R1.

Implementation approach

- Phase 1 (Months 1–2): flow export, windowed graphs, degree detectors, analyst training.
- Phase 2 (Months 2–3): communities, stronger baselines, visualisation in tickets, playbooks.
- Phase 3 (Months 3–5): optional graphlets/spectral features, tune thresholds, reassess risks every six months.

Limitations: large graphs are costly—use sampling and two-stage metrics. NAT may collapse users into one IP—prefer pre-NAT IDs when available. Thresholds need reviewed history—start conservative and tune. Graph alerts complement, not replace, IDS/EDR/firewall controls.

Governance note. Assign an owner for exporter health, baseline refresh and threshold review. Without ownership, graphs drift and false positives return. Quarterly tabletop exercises that replay a scan star and a DDoS star keep analysts fluent and keep management aware of residual risk after controls are deployed.

7. Conclusion

Summary of findings and recommendations

This case study examined network traffic analysis through graph theory in the Alliance University MCA format. Traditional volume thresholds and signatures underuse the relational nature of communication. Modelling hosts as vertices and flows as edges converts discrete structures into a practical cybersecurity lens for campus and enterprise networks.

Data collection showed NetFlow/IPFIX, Zeek and related metadata are enough to build windowed TDGs even when payloads are encrypted. Analysis showed DDoS, scanning, botnet C2, lateral movement and exfiltration leave distinctive topological footprints. Degree, centrality, communities and entropy provide explainable features; temporal baselines and IP ranking turn features into actionable SOC work.

The solution proposed a modular pipeline from sensors to SOC response, with controls mapped to Critical and High risks (R1–R5), a practical tool stack and a phased plan. Graph analysis does not eliminate cyber risk, but it reduces blind spots that volume-only monitoring leaves open—especially for distributed, encrypted and low-and-slow campaigns.

Summary of key findings

Area	Key finding
Problem	Host-centric and volume-only views miss coordinated structural attacks
Objectives	Six objectives cover collect → model → measure → map → detect → evaluate
Data	Flow metadata in fixed windows is enough to build useful TDGs
Analysis	Each major attack has a distinctive graph shape and feature set
Reason	Structural change beats byte thresholds for slow/distributed threats
Risk	1 Critical (scanning gap) and 4 High risks prioritise the solution
Solution	Windowed graphs → features → baseline → ranked IPs → SOC playbooks
Benefit	Faster detection, clearer triage, works with encrypted traffic

The findings table closes the case-study loop: every major claim is backed by earlier tables on sources, measures, attack mappings, scores and controls. That traceability is intentional for academic evaluation and for management read-through.

Final recommendations

- Approve flow collection and a windowed traffic-graph pipeline as a SOC project.
- Start with degree/new-edge detectors (quick wins), then add communities and robust baselines.
- Train analysts to read TDGs, centrality rankings and local attack subgraphs.
- Keep combining graph alerts with IDS, firewall and endpoint controls—defence in depth.
- Appoint an owner for graph-detection tuning; reassess risks every six months.
- Track the measurable targets in Objectives and report them to management.

Cyber risk cannot be removed completely, but it can be reduced. A graph-theoretic approach to traffic analysis—practical, interpretable and compatible with encryption—cuts blind spots left by volume-only and signature-only tools and strengthens both academic understanding and operational security practice.

Closing reflection. Across problem, objectives, data, analysis and solution, the same idea repeats: communication is a graph, attacks reshape that graph, and measurable structural features make those reshapes visible to people who must respond under time pressure. For MCA Semester I learners, that idea connects discrete structures to a credible cybersecurity workflow; for practitioners, it offers a phased path from NetFlow to ranked hosts and playbooks without waiting for decrypted payloads.

Final academic note. All numerical profiles and mini-scenarios in this document are illustrative for teaching and case-study design. They are structured to mirror the density and table-driven style of Alliance University sample case studies while remaining specific to Network Traffic Analysis Using Graph Theory.

References

Bridges et al., GraphPrints (arXiv:1602.01130). Traffic Dispersion Graph research. IP flow graph and entropy-based DDoS studies. Centrality and spectral methods for cyber situational awareness. NIST Cybersecurity Framework (CSF) 2.0 concepts for detect-and-respond. Illustrative campus/enterprise figures prepared for this MCA case study.